

Image Inpainting Through Graph Neural Network

Xingjian Li 1011563096

lestergg.li@mail.utoronto.ca

github: <https://github.com/LesterLi33/MIE1520-Image-Inpainting.git>

Abstract—This report will analyze the hypothesis that hybrid approach will improve the reconstruction of damaged regions in human face images, specifically using both Convolutional Neural Networks (CNNs) with Graph Neural Networks (GNNs). In contrast to traditional methods, usually including basic interpolation and convolutional neural networks (CNNs), the hybrid image inpainting framework often successfully reconstructs fine grained details or long range dependencies in facial structures. In this project, GNNs are incorporated to model spatial and contextual relationships across image patches by treating them as nodes in a graph. The proposed framework uses a CNN to extract local feature representations as a first step, then refined through GCN or GAT blocks to aggregate contextual information from distant regions. Experiments are conducted using CelebA dataset, and using PSNR, SSIM, and MSE as the evaluation metrics. The results showed that CNN+GAT model with three attention layers produces the most visually realistic inpainting. Although the quantitative scores were slightly lower compared to a U-Net autoencoder baseline. Overall, this project suggests that attention based GNNs can enhance perceptual quality by improving structural consistency, highlighting their potential for high fidelity image inpainting tasks.

Index Terms—Image inpainting, Graph Neural Networks (GNNs), Convolutional Neural Networks (CNNs), Patch-based features, CelebA dataset

I. INTRODUCTION

Image inpainting is one of the classic computer vision tasks. It is concerned with reconstructing the corrupted or missing regions of an image. The technique plays a critical role in many different real world applications, including but not limited to photo editing, the restoration of damaged historical images, object removal, surveillance footage enhancement, and medical imaging reconstruction. For example, image inpainting helps improve diagnostic accuracy as mentioned in 1, and it also aids in forensic image processing, including the restoration of degraded images and the removal of unwanted artifacts as shown in 2. In such cases, the ability to accurately and realistically fill in missing areas can significantly improve the interpretability of images. The challenge in this project aims to address is face image inpainting, specifically the accurate restoration of missing facial regions in images. Recent deep learning methods, especially those using CNNs, have made great progress in creating realistic-looking images for inpainting tasks. However, these methods still face some key challenges. Due to their fixed kernel size, CNNs analyze images using fixed receptive fields, which means they are very good at capturing textures, edges, and local shapes, but often struggle to capture global structures as mentioned in 3. To overcome CNNs' weaknesses, this project explores the integration of GNNs into the inpainting framework. Graph

Neural Networks proposed another strategy to connect different parts of an image by treating image patches or pixels as interconnected nodes in a graph. Thus, GNNs have the ability to model relational information between patches or pixels. The goal here is to create a hybrid model that combines the CNN's ability to extract local features with the GNN's ability to analyze the entire image as a connected system. By combining both advantages of GNN and CNN, the proposed model aims to fill in missing regions in a way that is both visually coherent and structurally consistent. The data set used in this project is the CelebFaces Attributes Dataset (CelebA), which was introduced by 4. It is a large-scale facial image dataset that contains more than 200,000 celebrity images. For face image inpainting, CelebA provides diversity and high resolution face images. Allowing the model to generalize well across diverse facial structures and appearances. Due to limitations in computational resources, only 8,000 images from the CelebA dataset were used for training and hyperparameter tuning. In this project, the research studied three model structures, U-net convolutional AutoEncoder 5, CNN+GCN and CNN+GAT. The model used PSNR 6 and SSIM 7 to evaluate their performance. Based on the results, Unet-convolutional AutoEncoder achieved the best performance, closely followed by CNN+GAT's results, CNN+GCN is slightly blurry in visual effects, but also achieves a relatively high PSNR and SSIM score. This shows using GNN+CNN structure has the ability to capture global structural information.

II. RELATED WORK

A. Traditional Methods for Image Inpainting

Image inpainting, especially face image inpainting, was popularly used in the identity verification, surveillance, photo restoration, and entertainment sectors. Due to its widespread usage, studies have been extensively analyzed involving this topic. Early methods largely relied on diffusion based and patch based techniques. One of the diffusion based approaches is using a diffusion model to propagate surrounding pixel information into missing regions 8. Another patch based approach is the development that fills holes by copying similar patches from known regions 9. Although these methods are effective in texture completion, they often fail to maintain semantic consistency in complex image structures.

B. CNN-Based Deep Learning Approaches

With the rise of deep learning, CNN became the mainstream method for image inpainting. One of the first deep learning frameworks for image inpainting using CNN was proposed in 3. This research designed an encoder decoder CNN that

learns to predict missing image regions based on the context from the surrounding area. Later on, a partial convolution based inpainting method was proposed in 10, which effectively handles irregularly shaped masks by conditioning the convolution operation only on valid pixels. Lastly, a UNet based convolutional autoencoder has been widely adopted for image inpainting tasks due to its ability to preserve both low-level textures and high-level semantic structures. In 5, an image inpainting method is introduced that utilizes an autoencoder architecture inspired by U-Net. The model is designed to reconstruct missing or corrupted regions in images by extracting and leveraging various features from the input data.

C. GAN Based Approaches

The emergence of Generative Adversarial Networks (GANs) has further promoted the development of deep learning in the direction of image inpainting. The generator produces realistic completions for missing image regions, and the discriminator cannot distinguish the generated part from the real ones. This adversarial strategy forces the generator to balance local features and overall structure. For instance, a GAN-based face completion model was proposed in 11, which not only restores missing facial parts but also maintains identity consistency through additional semantic losses. Other GAN models, such as EdgeConnect 12, further improve realism by using edge guidance and multi-step refinement. In general, GANs help models inpainting do more than just copy pixels—they create results that are both meaningful and visually convincing to the human eye. GAN was originally used as a comparison model in this report to measure the performance of the CNN + GGN structure model. However, due to its high requirements for GPU, this project used the U-Net Convolutional AutoEncoder as the comparison model.

III. METHODOLOGY

This project aims to resolve image inpainting problems by using hybrid models that integrate CNNs and GNNs, the implementation is conducted in python, including Pytorch and Pytorch Geometric.

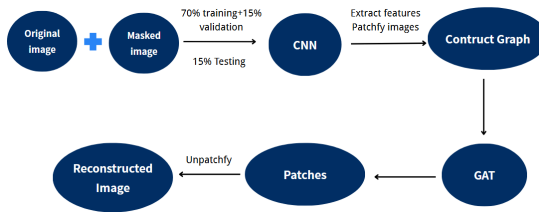


Fig. 1. Model Workflow

A. Data Preparation

The CelebA images were preprocessed by centering and cropping them (Cropped it 178x178), then resizing to 128x128 pixels. They were also normalized using standard PyTorch tools. To load the data, a custom CelebADataset class was

created. Because of limited computing resources, only 8,000 images from the dataset were used. These images were divided into training (0.7), validation (0.15), and testing (0.15) sets using the train-test split method.

To simulate the inpainting task, input images from the CelebA dataset are intentionally damaged by masking a central square region. A custom data set class, called InpaintingDataset, is defined to handle this masking process. It will return a masked version of the image along with the ground truth image. The mask size has been set to 30 x 30, meaning a 30 × 30 square at the center of the face image is set to zero (black). This setup provides paired training data in the form:

- **Input : Masked image (for model prediction)**
- **Target: Ground truth image (for loss calculation)**

B. Model Architecture

1) *CNN Encoder*: The CNN encoder comprises three convolutional layers, each of them followed by a ReLU activation function. In order to extract hierarchy visual features from the image. These layers progressively reduce the spatial resolution while increasing the channel of feature maps.

This downscaling process could be considered as dividing the image into patches. The number of patches depends on the patch size set by the user. For example, if the image size is 128 x 128 and patch size is 8 x 8, then the number of patches is 16 x 16. After feature extraction, a 1x1 convolution is used to project each patch into a lower dimensional feature. Those patches will be used as input node features for GNN.

2) *Graph Construction*: As each image is divided into a grid of patches, a grid based graph is constructed on top of them. Patches serve as graph nodes and edges connect spatially adjacent patches. Additionally, the model supports mini-batch processing by offsetting the edge indices per image and concatenating them. Thus, images from the same batch will be constructed to a giant graph. This is for controlling the parallel processing on GPU, improving training speed and avoiding GPU out of memory problems.

3) *GNN layers*: To achieve effective image restoration, this project leverages Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs) as core components of the inpainting framework. GCN is a good choice because it effectively captures local neighborhood information while maintaining computational efficiency 13. By aggregating and transforming the features of each node's neighbors through a normalized adjacency matrix, GCN enables smooth information propagation across the graph. The node features are updated according to the following formula:

$$H^{(l+1)} = \sigma \left(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)} \right)$$

GAT, on the other hand, allows the model to adaptively focus on the most relevant neighboring patches or pixels when reconstructing damaged or missing regions. It introduces learnable attention weights that highlight important relationships,

enabling better structural understanding and more effective feature propagation across the image. The node features are updated according to:

$$\mathbf{x}'_i = \alpha_{i,i} \Theta_s \mathbf{x}_i + \sum_{j \in \mathcal{N}(i)} \alpha_{i,j} \Theta_t \mathbf{x}_j$$

In this project, each model contains several GCN or GAT blocks, and the number of these blocks is determined by the user. A GCN block consists of a GCNConv layer followed by a linear (fully connected) layer used for the residual connection. The GCNConv layer takes d_{in} dimensional features as input then outputs d_{out} dimensional features after aggregating information from neighboring nodes based on the adjacency matrix. The residual layer transforms the original input $\mathbf{x}$ from d_{in} to d_{out} so that it can be added element-wise to the GCN output. The GAT block shares a similar structure. It includes a GATConv layer and followed by a fully connected layer, which is used for the residual connection. The GATConv layer takes d_{in} dimensional features as input and outputs d_{out} dimensional features by computing attention weighted aggregations over neighbor nodes defined by adjacency matrix. Attention coefficients are learned during training. The model is able to assign greater importance to more relevant neighbors. Same as above, during the forward pass, the GAT output and the residual transformed input are added element-wise.

4) *Decoder*: After several rounds of GNN-based message passing, the output node embeddings are passed through a fully connected decoder that reconstructs the original RGB patch content. Each patch produces a flattened vector of size $3 \times \text{patch_size} \times \text{patch_size}$. Therefore, the overall output shape is $\text{batch_size} \times \text{num_patches} \times 3 \times \text{patch_size} \times \text{patch_size}$.

5) *Evaluation*: The model's performance is measured using SSIM (Structural Similarity Index) and PSNR (Peak Signal to Noise Ratio), two widely used metrics for evaluating the quality of image inpainting. SSIM measures the similarity between two images based on perceived changes in luminance, contrast, and structure. Which better align with human visual perception. It ranges from -1 to 1 .

Formula:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

PSNR is another metric that compares the maximum possible power of a signal to the power of corrupting noise. It is calculated using the mean squared error (MSE) between the original and restored images. Higher PSNR values indicate better quality.

Formula:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

C. Model Comparison

The U-Net Convolutional AutoEncoder is one of the widely used models in image reconstruct tasks. This report uses it as the comparison model to evaluate the performance of CNN+GNN models.

Key features of the U-Net Convolutional AutoEncoder

- **Input**: Masked image.
- **Encoder**: Compresses the input image through convolution and max pooling layers, progressively reducing spatial resolution while increasing feature depth.
- **Bottleneck**: Learns compact high level representations of the image.
- **Decoder**: Uses transposed convolutions (upsampling) and skip connections to reconstruct the image, combining low level details from the encoder with high level features.
- **Output**: An restored image of the same size as the input.

D. Hyperparameter Tuning

In addition to the conventional hyperparameter adjustments (such as learning rate, number of epochs, weight decay etc.), the important hyperparameter adjustments for this project are as follows:

1) *Replace GCN by GAT*: Although the GCN based model achieved a PSNR of 30.67 and an SSIM of approximately 0.93, the visual quality of the restored images was unsatisfactory. To the human eye, there were noticeable color differences between the restored and original regions, and the edges of the patches were clearly visible. There have been several attempts to address this problem, including increasing the number of GCN blocks, adding positional encoding, replacing the CNN encoder with ResNet18, and switching from MSE loss to L1 loss. However, these adjustments yielded limited improvements. The final approach replacing GCN blocks by GAT blocks. GAT assigns dynamic attention weights to each neighboring patch during message passing, allowing the model to focus more on important or semantically relevant regions. As a result, the output images appeared visually sharper and more natural, even though the pixel wise error increased slightly (leading to lower PSNR and SSIM scores).

2) *Adjust the number of GAT block*: After finalizing the CNN+GAT structure and tuning conventional hyperparameter, I experimented with different numbers of GAT layers. The best performance (balancing PSNR, SSIM, and visual quality) was achieved using a 3 layer CNN followed by 3 GATConv layers, optimized with Adam and trained using MSE loss.

IV. RESULTS

A. Performance comparison

Model	PSNR	SSIM	Test MSE
CNN+GCN	30.67	0.93	0.000865
CNN+GAT (3 block)	30.41	0.92	0.000919
CNN+GAT (4 block)	30.16	0.92	0.000972
CNN+GAT (5 block)	29.81	0.91	0.001053
U-Net Convolutional AutoEncoder	34.48	0.9804	0.000023

B. Sample output



Fig. 2. CNN+GAT



Fig. 3. CNN+GCN

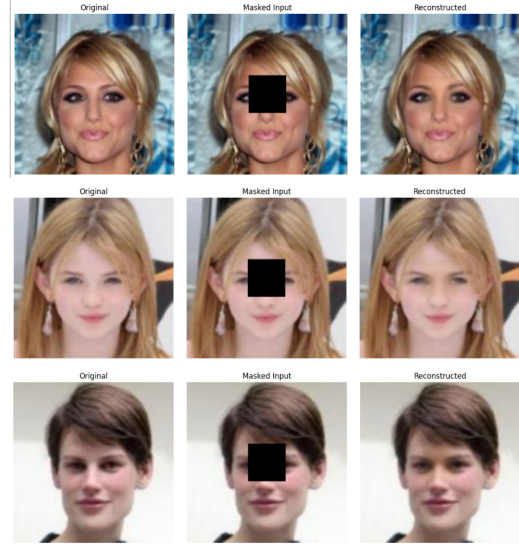


Fig. 4. U-net Convolutional AutoEncoder (comparison model)

C. Discussion

The results show that the U-Net Convolutional AutoEncoder outperforms all other models across all three evaluation metrics: PSNR, SSIM, and Test MSE. It achieves the highest PSNR (34.48), SSIM (0.9804) with lowest MSE (0.000023). Indicating it generates reconstructions that are both sharp and perceptually close to the ground truth and have the minimal pixel-wise error. Among the CNN + GNN based models, the CNN + GCN model seems has the best performance, it achieved PSNR of 30.67, SSIM of 0.93, and a relatively low MSE of 0.000865. However, from a sensory perspective, the output of CNN+GCN is not well restored to the naked eye. Compared with the CNN+GAT structure, the traces of filling can be clearly seen. Therefore, the performance of CNN+GAT structure is better than that of CNN+GCN structure. Combining CNN and GNN can fully utilize the advantages of both. This structure is particularly suitable for tasks that require structural consistency and semantic coherence, such as face image restoration and scene completion. Experimental results also show that although the reconstruction accuracy of the current CNN+GNN architecture is not as good as that of classic models such as U-Net AutoEncoder, it has great potential in restoring complex structures and global consistency. The device limitations of the project are not sufficient to support the exploration of more complex CNN+GNN structures. The theoretical optimization direction is mentioned in the conclusion section.

V. CONCLUSION

This project explored hybrid CNN+GNN architectures (using GCN and GAT layers) for image inpainting. Among the GNN-based approaches, the 3 layer CNN + 3 GAT blocks model produced the best visual quality. Although it did not achieve the highest PSNR or SSIM scores, the experimental results highlight the potential of GNNs in image inpainting

tasks is a promising direction for future research. To further improve the performance of the CNN+GAT model, several enhancements could be considered. First, using smaller patch sizes or even individual pixels as graph nodes may enable better spatial reasoning and more accurate reconstructions. Particularly in areas with complex textures or sharp edges. Second, integrating a GAN into the CNN+GNN framework could increase the realism of the restored images. The adversarial loss from the GAN would encourage the model to generate sharper, more natural looking results by reducing blur and artifacts.

REFERENCES

- [1] J. C. Santos, H. Tomás Pereira Alexandre, M. Seoane Santos, and P. Henriques Abreu, "The role of deep learning in medical image inpainting: A systematic review," *ACM Trans. Comput. Healthcare*, Jan. 2025, just Accepted. [Online]. Available: <https://doi.org/10.1145/3712710>
- [2] V. V.V., M. V.I., S. E.A., M. S.V., and C. R., "Digital inpainting with applications to forensic image processing," *Electronic Imaging*, vol. 28, no. 7, pp. 1–1, 2016. [Online]. Available: <https://library.imaging.org/ei/articles/28/7/art00010>
- [3] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2536–2544.
- [4] J. Li, "Celeba dataset," <https://www.kaggle.com/datasets/jessicali9530/celeba-dataset>, 2023, accessed: 2025-04-13.
- [5] M. H. Givkashi, M. Hadipour, A. PariZanganeh, Z. Nabizadeh, N. Karimi, and S. Samavi, "Image inpainting using autoencoder and guided selection of predicted pixels," 2021. [Online]. Available: <https://arxiv.org/abs/2112.09262>
- [6] A. Horé and D. Ziou, "Image quality metrics: Psnr vs. ssim," in *2010 20th International Conference on Pattern Recognition*, 2010, pp. 2366–2369.
- [7] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [8] M. Bertalmío, G. Sapiro, V. Caselles, and C. Ballester, "Image inpainting," 01 2000, pp. 417–424.
- [9] A. Criminisi, P. Perez, and K. Toyama, "Region filling and object removal by exemplar-based image inpainting," *IEEE Transactions on Image Processing*, vol. 13, no. 9, pp. 1200–1212, 2004.
- [10] G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao, and B. Catanzaro, "Image inpainting for irregular holes using partial convolutions," 2018. [Online]. Available: <https://arxiv.org/abs/1804.07723>
- [11] Y. Li, S. Liu, J. Yang, and M.-H. Yang, "Generative face completion," 2017. [Online]. Available: <https://arxiv.org/abs/1704.05838>
- [12] K. Nazeri, E. Ng, T. Joseph, F. Z. Qureshi, and M. Ebrahimi, "Edgeconnect: Generative image inpainting with adversarial edge learning," 2019. [Online]. Available: <https://arxiv.org/abs/1901.00212>
- [13] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>